Tutorial Clarity v152 — Sync Architecture Analysis Report

Date: May 11, 2026
Analyst: Abacus AI Agent
Larry's Question: "Does George look at the YouTube video time before he starts the time for the video and TTS?"

Executive Summary

Short answer: Yes, George DOES look at YouTube video time — but the system has TWO competing sync architectures that fight each other, and the TTS audio timeline is built on SYNTHETIC timing that doesn't match the actual video.

The "252 seconds and stuttering" issue stems from a fundamental architectural conflict: the TTS audio segments are given **fabricated start times** based on word-count proportional scaling, while the sync engine tries to match them to **real YouTube video time**. These two timelines inevitably diverge.

---

1. The Two Competing Architectures

Architecture A: `useClarifyAudio.ts` + `clarifyAudioEngine.ts` + `audioBufferManager.ts`

This is the **older architecture**. It uses:
- `ClarifyAudioEngine` — word-matching/Levenshtein sync engine
- `AudioBufferManager` — browser `SpeechSynthesis` TTS
- Sequential playback from `playFromTime(time)`

Architecture B: `useAudioTranslation.ts` (v152) + `useAudioClarification.ts` (v129)

This is the **newer, active architecture**. It uses:
- "George the Video Editor" — a 100ms/500ms interval loop that polls YouTube player time
- OpenAI TTS via `/api/multi-voice-tts` API route
- `HTMLAudioElement` playback with preloading
- Segment-index-based sequential playback

⚠️ **CRITICAL FINDING**: `page.tsx.broken` (the main page) renders `ClarifyAudioPanel` which internally uses **Architecture A** (`useClarifyAudio`), BUT the `ClarifyAudioPanel` component shown in the uploaded file calls `useClarifyAudio(videoId, currentTime)` — a different signature than what the hook actually exports. The hook expects `options: UseClarifyAudioOptions`. This is a **type mismatch** that would cause the hook to fail silently.

---

2. Does George Check YouTube Time Before Starting TTS?

YES — George checks video time. Here's exactly how:

George's Video Time Resolution (`getVideoTime()` in `useAudioTranslation.ts`, ~line 787):

```
const getVideoTime = useCallback((reason: string = 'runtime'): number => {
    let resolved = Number.isFinite(currentTimeRef.current) ? currentTimeRef.current :
0;
    let source: 'callback' | 'youtube' | 'element' | 'ref' = 'ref';

    // Priority 1: Callback from parent component
    const callbackTime = Number(getVideoCurrentTimeRef.current?.());
    if (Number.isFinite(callbackTime) && callbackTime >= 0 && ...) {
        resolved = callbackTime;
        source = 'callback';
    } else {
        // Priority 2: Direct YouTube player API
        const ytTime = Number(getYouTubePlayerCurrentTime());
        if (Number.isFinite(ytTime) && ytTime >= 0 && ...) {
            resolved = ytTime;
            source = 'youtube';
        } else {
            // Priority 3: HTML5 <video> element
            const videoEl = activeVideoEl ?? getControllableVideoElement();
            const elementTime = Number(videoEl?.currentTime);
            ...
        }
    }
    return resolved;
}, [...]);
```

George tries **four sources** to get the current video time:

1. `getVideoCurrentTime` **callback** from parent component
2. `getYouTubePlayerCurrentTime()` — scans `window.__TC_ACTIVE_YT_PLAYER__`, `window.player`, etc. for `.getCurrentTime()`
3. **Direct** `<video>` **element** — `document.querySelector('video').currentTime`
4. **Fallback** — `currentTimeRef.current` (the last known value)

George's YouTube Player Discovery (`getYouTubePlayerCurrentTime()`, ~line 748):

```
const getYouTubePlayerCurrentTime = useCallback((): number | null => {
    const w = window as any;
    const candidates = [
        w.__TC_ACTIVE_YT_PLAYER__,
        w.__YT_PLAYER__,
        w.youtubePlayer,
        w.player,
        w.ytPlayer,
        w.ytplayer?.player,
    ];
    for (const candidate of candidates) {
        if (!isPlayable(candidate)) continue;
        const t = Number(candidate.getCurrentTime());
        if (Number.isFinite(t) && t >= 0) {
            youtubePlayerCacheRef.current = candidate;
            return t;
        }
    }
    return null;
}, []);
```

BUT — There's a Critical Gap

The `page.tsx.broken` does NOT expose the player to `window.__TC_ACTIVE_YT_PLAYER__`!

Looking at the YouTube player initialization in `page.tsx.broken` (line 256):

```
playerRef.current = new (window as any).YT.Player('youtube-player', {
    videoId: videoId,
    playerVars: { autoplay: 0, controls: 0, ... },
    events: {
        onReady: (event: any) => {
            playerReadyRef.current = true;
            setDuration(event.target.getDuration());
        },
        onStateChange: (event: any) => { ... }
    }
});
```

The player is stored in `playerRef.current` but is **never** exposed to any global variable that George can find. George searches for `window.__TC_ACTIVE_YT_PLAYER__`, `window.player`, `window.ytPlayer`, etc. — **none of these are set by page.tsx.broken**.

The `currentTime` state variable IS passed to `ClarifyAudioPanel`:

```
<ClarifyAudioPanel
    videoId={videoId}
    currentTime={currentTime}  // ← This comes from the 100ms polling interval
    duration={duration}
    isPlaying={isPlaying}
    onPauseVideo={pauseVideo}
    onResumeVideo={playVideo}
/>
```

This `currentTime` is polled every 100ms (line 336-348):

```
useEffect(() => {
    const interval = setInterval(() => {
        if (playerRef.current && playerReadyRef.current) {
            try {
                setCurrentTime(playerRef.current.getCurrentTime());
            } catch (e) { }
        }
    }, 100);
    return () => clearInterval(interval);
}, []);
```

So **the parent page does poll** `getCurrentTime()`, but the question is whether this value actually reaches the TTS engine.

---

3. The Root Cause: Synthetic Timeline vs. Real Video Timeline

The 252-Second Problem

The `adaptSegmentsForAudio()` function in `useAudioClarification.ts` (line 403-625) creates **a completely synthetic timeline**:

```
function adaptSegmentsForAudio(segments, targetLanguage) {
    // STEP 1: Calculate raw duration based on WORD COUNT (not video timing)
    const rawDurations = segments.map(seg => {
        const wordCount = text.split(/\s+/).filter(w => w.length > 0).length;
        const rawDuration = Math.max((wordCount / 150) * 60, 0.5); // 150 WPM
        return rawDuration;
    });

    // STEP 2: Scale durations to fit total video estimate
    const totalRawDuration = rawDurations.reduce((sum, d) => sum + d, 0);
    const scaleFactor = totalDurationEstimate / Math.max(totalRawDuration, 0.1);

    // STEP 3: CUMULATIVE start times (not from YouTube!)
    let cumulativeTime = 0;
    const adapted = segments.map((seg, index) => {
        const scaledDuration = rawDurations[index] * scaleFactor;
        const start = cumulativeTime;  // ← SYNTHETIC! Not from YouTube
        cumulativeTime += scaledDuration;
        return { text: seg.text, start: start, duration: scaledDuration, ... };
    });
}
```

This is the smoking gun. The TTS segments get timestamps like:
- Segment 0: 0.00s - 2.34s
- Segment 1: 2.34s - 5.67s
- Segment 2: 5.67s - 8.12s
- ...etc, accumulating linearly

But the **actual YouTube captions** might have timing like:
- Caption 0: 0.50s - 3.20s (speaker starts late)

- Caption 1: 5.10s - 7.80s (gap between speakers)
- Caption 2: 7.80s - 12.40s (longer segment)

The synthetic timeline assumes continuous speech with NO gaps, while real video has pauses, transitions, music breaks, etc. Over a 10-minute video, this drift accumulates dramatically.

Why 252 Seconds Specifically?

If the video is ~10 minutes (600s) and the transcript has gaps totaling ~60% of the video (common for tutorials with pauses, screen recordings, etc.), the TTS audio covers only ~252 seconds of actual speech content. The TTS plays all 252 seconds of speech continuously, but the video timeline spreads those same words across 600 seconds.

The Stuttering

The stuttering comes from George's sync loop (in `useAudioTranslation.ts`) which runs every 100-500ms and tries to reconcile the two diverging timelines:

```
// George checks: "Video is at 45.2s, which segment should play?"
const getSegmentIndexForVideoTime = (videoTime) => {
    for (let i = 0; i < segmentTimesMap.length; i++) {
        if (videoTime >= seg.startTime && videoTime < seg.endTime) {
            return i;  // Found matching segment
        }
    }
    // Falls through to "closest" fallback...
};
```

When video jumps (due to buffering, user seeking, or natural gaps), George detects a mismatch and:
1. Stops current audio
2. Seeks to "correct" segment
3. Starts playing new segment
4. By the time that starts, video has moved again
5. Repeat → **stutter**

The jump alignment logic has a 2.5-second threshold (line 118):

```
const JUMP_ALIGNMENT_THRESHOLD_SECONDS = 2.5;
const JUMP_COOLDOWN_MS = 2000;
```

But when the drift exceeds 2.5s (which happens quickly with synthetic timing), George keeps triggering jumps, causing the stutter.

---

4. Sync Initialization Flow (What Happens When Play is Pressed)

Step-by-Step Flow:

```
User clicks "Start Clarification" in ClarifyAudioPanel
    │
    ▼
ClarifyAudioPanel.handleStart() → actions.startClarification(videoId, language)
    │
    ▼
useClarifyAudio.start(startTime = 0)  ← DEFAULT startTime IS ZERO!
    │
    ├─ POST /api/process-video { videoId, option: 2, targetLanguage }
    │     └─ Returns transcript segments
    │
    ├─ engine.setOriginalTranscript(transcript)
    │
    ├─ bufferManager.initialize(clarifiedSegments, targetLanguage)
    │
    ├─ bufferManager.prebufferFrom(startSegmentIndex)
    │     └─ startSegmentIndex = segments.findIndex(s => s.start >= startTime)
    │     └─ Since startTime = 0, starts from segment 0 ALWAYS
    │
    └─ bufferManager.playAtTime(startTime)  ← PLAYS FROM startTime = 0!
```

⚠️ **CRITICAL BUG**: `start()` is called with `startTime = 0` by default. It does NOT check `currentTime` (the YouTube player's current position). Even though `currentTime` is passed as a prop to `ClarifyAudioPanel`, it's never used to initialize the start position:

```
// ClarifyAudioPanel.tsx line 34-36:
const handleStart = async () => {
    await actions.startClarification(videoId, selectedLanguage);
    // ← currentTime is NEVER passed here!
};
```

Meanwhile, in the NEWER architecture (`useAudioClarification.ts`), the `start()` function does accept `startTime` :

```
// useClarifyAudio.ts line 469:
const start = useCallback(async (startTime: number = 0) => {
    // startTime DEFAULTS TO ZERO
    // ...
    await bufferManager.playAtTime(startTime);  // Always starts at 0!
}, [...]);
```

---

5. The Three Timelines That Don’t Align

Timeline	Source	Behavior
YouTube Video	`player.getCurrentTime()`	Real wall-clock time with gaps, pauses, buffering
TTS Audio	Synthetic cumulative timing	Continuous speech, no gaps, word-count proportional
Caption/Word Highlighting	`currentTime` state from 100ms poll	Follows YouTube time but displayed against synthetic phrase timing

The Mismatch in Pictures:

```
YouTube Video Timeline:
[speech][gap][speech][long gap][speech][music][speech]
0s     10s   20s     30s       50s     60s    70s

TTS Audio Timeline (synthetic):
[speech][speech][speech][speech]
0s     10s     20s     30s     ← ONLY 30s of audio for 70s of video!

Word Highlighting:
Uses phrase timing from adaptSegmentForPhrases() → uses synthetic times
So highlighting at video time 50s looks for phrases at 50s synthetic time
But synthetic timeline only goes to 30s → NO MATCH → blank highlights
```

---

6. Summary of Root Causes

Root Cause #1: TTS Start Always at 0:00

The `start()` function defaults `startTime = 0` and `ClarifyAudioPanel.handleStart()` never passes the current YouTube position. If the user is 3 minutes into a video, TTS starts from the beginning.

Root Cause #2: Synthetic Timeline

`adaptSegmentsForAudio()` builds a synthetic cumulative timeline based on word count rather than preserving original YouTube caption timestamps. This creates an alternate reality that progressively diverges from the video.

Root Cause #3: Two Competing Architectures

`page.tsx.broken` uses `ClarifyAudioPanel` → `useClarifyAudio` (Architecture A), while the more complete sync logic lives in `useAudioTranslation` (Architecture B / George). George’s sophisticated video-time polling never gets used because the wrong hook is wired up.

Root Cause #4: Player Not Exposed to George

Even if Architecture B were active, `page.tsx.broken` stores the YT player in a local ref and never exposes it via `window.__TC_ACTIVE_YT_PLAYER__` or the `getVideoCurrentTime` callback, so George's `getVideoTime()` falls back to `currentTimeRef.current = 0`.

Root Cause #5: Stutter from Over-Correction

George's 100ms sync loop detects drift → triggers segment jump → cooldown → detects drift again → stutter loop. The 2.5s threshold + 2s cooldown constants aren't tuned for the synthetic timeline's massive drift.

---

7. Recommended Fixes

Fix 1: Pass `currentTime` to start()

```
// ClarifyAudioPanel.tsx — use the currentTime prop
const handleStart = async () => {
    await actions.startClarification(videoId, selectedLanguage, currentTime);
    //                                                          ^^^^^^^^^^
};
```

Fix 2: Preserve Original YouTube Timestamps

In `adaptSegmentsForAudio()`, use the original caption `start` times instead of building synthetic cumulative times:

```
// INSTEAD of cumulativeTime approach:
const adapted = segments.map((seg, index) => {
    const timing = resolveSegmentTiming(seg, index, ...);
    return {
        text: seg.text,
        start: timing.start,      // ← ORIGINAL YouTube timestamp
        duration: timing.duration, // ← ORIGINAL duration
    };
});
```

Fix 3: Expose Player to George

```
// In page.tsx, after player is ready:
onReady: (event: any) => {
    playerReadyRef.current = true;
    setDuration(event.target.getDuration());
    (window as any).__TC_ACTIVE_YT_PLAYER__ = event.target;
    //                                        ^^^^^^^^^^^^^
},
```

Fix 4: Unify on One Architecture

Pick Architecture B (George/useAudioTranslation) and wire it in properly with correct props including `getVideoCurrentTime`, `setVideoPlaybackRate`, and `isVideoPlaying`.

---

8. Code Snippet Evidence

Evidence: TTS starts at 0 regardless of video position

File: `useClarifyAudio.ts`, line 469

```
const start = useCallback(async (startTime: number = 0) => {
    // ^^^^^^^^^^^^^^^^ DEFAULT IS ZERO
```

Evidence: Synthetic timeline construction

File: `useAudioClarification.ts`, line 460-467

```
let cumulativeTime = 0;
const adapted = segments.map((seg, index) => {
    const scaledDuration = rawDurations[index] * scaleFactor;
    const start = cumulativeTime;  // SYNTHETIC
    cumulativeTime += scaledDuration;
    // ...
});
```

Evidence: George DOES check YouTube time

File: `useAudioTranslation.ts`, line 748-785

```
const getYouTubePlayerCurrentTime = useCallback((): number | null => {
    // Scans window globals for YouTube player
    // Calls candidate.getCurrentTime()
    // Returns real video time
}, []);
```

Evidence: Player NOT exposed to George

File: `page.tsx.broken`, line 256-299

```
playerRef.current = new (window as any).YT.Player('youtube-player', {
    // Player stored in local ref only
    // Never assigned to window.__TC_ACTIVE_YT_PLAYER__
});
```

Report generated by analyzing uploaded v152 source files. All line numbers reference the uploaded file versions.